# DBMS Complete

16 April 2026 17:20

## TODOS:

- Introduction to Data, Database, and DBMS
- File System vs DBMS and why DBMS matters
- Three-schema architecture and data independence
- Data models: Hierarchical, Network, Relational, NoSQL
- ER Model — entities, attributes, relationships, cardinality, specialization, generalization
- Relational model terminology and properties
- All types of keys: Primary, Foreign, Candidate, Super, Composite, Surrogate
- Complete SQL: DDL, DML, DCL, TCL, DQL
- Joins: Inner, Left, Right, Full, Cross, Self
- Subqueries and correlated subqueries
- Views, Stored Procedures, and Triggers
- Normalization: 1NF, 2NF, 3NF, BCNF, 4NF and Functional Dependencies
- ACID properties and Transaction management
- Concurrency control: Locking, 2PL, Deadlocks, Timestamps
- Indexing: B-Tree, Hash, Dense, Sparse, Clustered, Non-Clustered
- File organization methods
- Relational Algebra
- NoSQL databases and the CAP Theorem
- Database Recovery and Write-Ahead Logging
- Advanced SQL: Window Functions, CTEs, CASE
- Complete database design example (E-Commerce)
- 20 most asked interview questions with complete answers

## WHO IS THIS FOR?

**WHO IS THIS VIDEO FOR?**

This video is for:

- Students preparing for university exams (BCA, MCA, B.Tech, BSc CS)
- Freshers and developers preparing for technical interviews at product and service-based companies
- Anyone who has been avoiding databases and wants to understand them in one sitting
- Backend developers who write SQL daily but have never understood what is happening under the hood

**WHY SHOULD YOU STUDY DBMS?**

Every application you use — Instagram, Amazon, your bank, your hospital — stores data. That data has to be organized, retrieved, and protected efficiently. DBMS is the engine that makes that possible.

In interviews at companies like TCS, Infosys, Wipro, Amazon, Flipkart, and every startup, DBMS questions appear in almost every technical round. It is one of the highest-scoring subjects because the concepts are fixed, the questions are predictable, and preparation pays off directly.

This video covers every major topic from scratch — no prior knowledge assumed. By the end, you will understand how data is stored, how to query it, how it stays consistent, and how systems handle millions of users without losing a single record.

# PART 1 — INTRODUCTION TO DBMS

## 1.1 What is Data?

Data is a collection of raw, unorganized facts and figures. On its own, data has no meaning.

**Example:**

9876543210, Rahul, Delhi, 25

This is data. Without context, you do not know what it represents.

9876543210, Rahul, Delhi, 25
This is data. Without context, you do not know what it represents.

## 1.2 What is Information?

When data is processed, organized, and given context, it becomes information.
**Example:**
Name: Rahul
Age: 25
City: Delhi
Phone: 9876543210
Now it is meaningful. This is information.

## 1.3 What is a Database?

A database is an organized collection of structured data stored electronically so that it can be easily accessed, managed, and updated.
Think of it as a digital filing cabinet where every drawer is labeled, every file is indexed, and you can find anything in seconds.
**Real-world examples:**

- Gmail stores your emails, contacts, and settings in a database
- Amazon stores every product, order, and customer in a database
- Your college stores student records, marks, and attendance in a database

## 1.4 What is a DBMS?

A **Database Management System (DBMS)** is a software system that acts as an interface between the user and the database. It allows users to create, read, update, and delete data while managing security, concurrency, and integrity behind the scenes.
**Definition:**

A DBMS is a collection of programs that enables users to create and maintain a database.

**Examples of DBMS software:**

- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- MongoDB (NoSQL)
- SQLite

## 1.5 File System vs DBMS

Before DBMS, data was stored in flat files. Here is why that was a problem and why DBMS was built to fix it.

| Problem in File System | How DBMS Solves It |
|---|---|
| Data Redundancy — same data stored in multiple files | Normalization removes duplicate data |
| Data Inconsistency — one file updated, other not | Single source of truth, constraints enforce consistency |
| Difficult Data Access — need custom programs to retrieve data | SQL allows flexible queries without programming |
| No Security — any program could access any file | Access control, user roles, and permissions |
| No Atomicity — partial writes cause corruption | Transactions ensure all-or-nothing operations |
| No Concurrent Access — two users editing same file causes errors | Concurrency control with locking and timestamps |

**Scenario:** Imagine a hospital using text files. Doctor updates patient blood group in one file. Nurse reads from a different file that still shows the old blood group. The patient gets the wrong transfusion. This is the data inconsistency problem that DBMS prevents.

reads from a different file that still shows the old blood group. The patient gets the wrong transfusion. This is the data inconsistency problem that DBMS prevents.

## 1.6 Advantages of DBMS

- **Reduced Data Redundancy** — Data is stored once and referenced everywhere
- **Data Integrity** — Rules (constraints) prevent invalid data from entering the system
- **Data Security** — Role-based access ensures only authorized users see sensitive data
- **Data Abstraction** — Users interact with data without worrying about how it is physically stored
- **Concurrent Access** — Multiple users can read and write simultaneously without conflict
- **Backup and Recovery** — Automatic mechanisms restore data after crashes
- **Data Independence** — Changes to storage structure do not affect application code

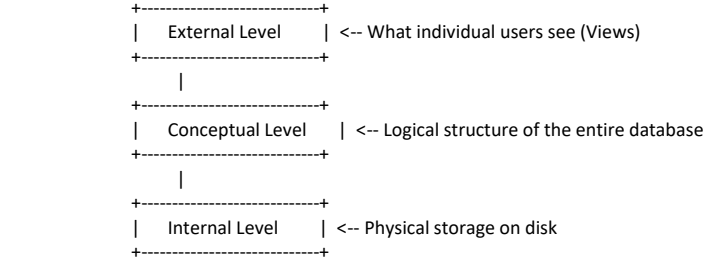
## 1.7 Disadvantages of DBMS

- High initial cost (hardware, software, training)
- Complexity — requires skilled database administrators
- Performance overhead — for very simple applications, a flat file might be faster
- Single point of failure if not configured with replication

# PART 2 — DBMS ARCHITECTURE

## 2.1 Three-Schema Architecture (ANSI/SPARC Architecture)

The three-schema architecture separates the database into three levels so that changes at one level do not affect the other levels. This is the foundation of **data independence**.




**External Level (View Level)**

- Each user or application sees only a portion of the database relevant to them
- Example: A bank teller sees account balance and transaction history. They do not see internal credit scores or loan algorithms

**Conceptual Level (Logical Level)**

- Describes what data is stored and the relationships between data
- Defined using tables, columns, data types, and constraints
- Example: The conceptual schema defines that a Customer table exists with columns customer_id, name, email

**Internal Level (Physical Level)**

- Describes how data is physically stored on disk
- Deals with file organization, indexes, compression, and storage allocation
- Example: Customer data is stored in a B-tree index on disk at block 142

## 2.2 Data Independence

Data independence means that changes at one level of the architecture do not require changes at other levels.

**Logical Data Independence**

- The ability to change the conceptual schema without affecting the external schema (user views)
- Example: You add a new column phone_number to the Customer table. Existing applications that do not use this column continue working without modification

**Logical Data Independence**

- The ability to change the conceptual schema without affecting the external schema (user views)
- Example: You add a new column phone_number to the Customer table. Existing applications that do not use this column continue working without modification

**Physical Data Independence**

- The ability to change the internal schema (physical storage) without affecting the conceptual schema
- Example: The DBA moves data from an HDD to an SSD or changes the indexing strategy. The logical structure and application queries remain unchanged

## 2.3 DBMS Architecture — 1-Tier, 2-Tier, 3-Tier

**1-Tier Architecture**

- The database, the application, and the user interface are all on the same machine
- Used in personal applications and local development
- Example: SQLite database used in a local desktop application

**2-Tier Architecture**

- The client (application) communicates directly with the database server
- Example: A Java application on a developer's laptop connecting directly to a MySQL server
- Problem: If 1000 users connect directly to the database, the database server is overloaded

**3-Tier Architecture (Industry Standard)**

- Three layers: Presentation (frontend), Application (business logic), and Database
- The application server acts as middleware between the client and the database
- Example: User browser → React frontend → Node.js server → PostgreSQL database

[ Browser / Mobile App ]
|
[ Application Server (Node.js / Spring / Django) ]
|
[ Database Server (MySQL / PostgreSQL / Oracle) ]

# PART 3 — DATA MODELS

## 3.1 What is a Data Model?

A data model defines how data is structured, stored, and accessed in a database. It is the blueprint of the database.

## 3.2 Types of Data Models

**Hierarchical Model**

- Data is organized in a tree structure (parent-child relationship)
- Each child has exactly one parent
- Example: File system directory structure
- Problem: Cannot represent many-to-many relationships

**Network Model**

- An extension of the hierarchical model
- A child can have multiple parents
- Uses pointers to link records
- Example: Early airline reservation systems

**Relational Model (Most Widely Used)**

- Data is organized in tables (relations)
- Relationships are defined through keys, not pointers
- Proposed by Edgar F. Codd in 1970
- Examples: MySQL, PostgreSQL, Oracle

**Object-Oriented Model**

- Data is stored as objects (like in object-oriented programming)
- Supports inheritance, encapsulation
- Examples: ObjectDB

**Document Model (NoSQL)**

- Data is stored as documents (JSON/BSON format)
- Example: MongoDB

# PART 4 — ENTITY-RELATIONSHIP (ER) MODEL

## 4.1 What is the ER Model?

The ER (Entity-Relationship) Model is a conceptual data model that describes the structure of a database using entities, attributes, and relationships. It was introduced by Peter Chen in 1976. It is the first step in database design — before writing a single line of SQL, you design the ER diagram.

## 4.2 Entity

An entity is a real-world object or concept that has existence and about which data is stored.

**Types of Entities:**

**Strong Entity**

- Exists independently
- Has its own primary key
- Represented by a rectangle
- Example: Student, Employee, Product

**Weak Entity**

- Cannot exist without a strong entity
- Does not have its own primary key — uses a partial key combined with the parent entity's key
- Represented by a double rectangle
- Example: Dependent (depends on Employee), Order Item (depends on Order)

## 4.3 Attribute

An attribute is a property or characteristic of an entity.

**Types of Attributes:**

| Type | Description | Example |
|---|---|---|
| Simple | Cannot be divided further | age, salary |
| Composite | Can be divided into sub-attributes | full_name → first_name + last_name |
| Single-valued | Holds one value | date_of_birth |
| Multi-valued | Holds multiple values | phone_numbers (a person can have more than one) |
| Derived | Calculated from another attribute | age derived from date_of_birth |
| Key | Uniquely identifies an entity | student_id, email |
| Null | May have no value | middle_name |

## 4.4 Relationship

A relationship describes how two entities are associated.

**Example:** Student **enrolls in** Course

**Degree of Relationship:**

- Unary (degree 1) — An entity related to itself. Example: Employee **manages** Employee
- Binary (degree 2) — Two entities. Example: Student **enrolls in** Course
- Ternary (degree 3) — Three entities. Example: Doctor **prescribes** Medicine **to** Patient

## 4.5 Cardinality (Mapping Constraints)

Cardinality defines the number of instances of one entity that can be associated with instances of another entity.

**One-to-One (1:1)**

- One entity instance is associated with exactly one instance of another
- Example: One Person has one Passport. One Passport belongs to one Person

**One-to-Many (1:N)**

- One entity instance is associated with multiple instances of another
- Example: One Department has many Employees. One Employee belongs to one Department

**Many-to-One (N:1)**

- Multiple instances of one entity are associated with one instance of another
- Same as 1:N but from the other perspective

**Many-to-Many (M:N)**

- Multiple instances of both entities can be associated with each other
- Example: One Student can enroll in many Courses. One Course can have many Students
- Implementation: Requires a junction/bridge table in the relational model

**Many-to-Many (M:N)**

- Multiple instances of both entities can be associated with each other
- Example: One Student can enroll in many Courses. One Course can have many Students
- Implementation: Requires a junction/bridge table in the relational model

## 4.6 Participation Constraints

**Total Participation**

- Every entity in the entity set must participate in at least one relationship
- Represented by double lines in the ER diagram
- Example: Every Employee must work in a Department (no employee can exist without a department)

**Partial Participation**

- Not every entity needs to participate in the relationship
- Represented by single lines
- Example: Not every Employee manages a Department

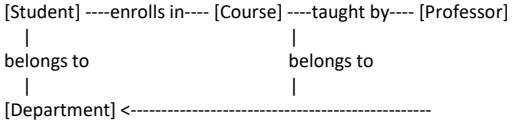
## 4.7 ER Diagram — Real Example

**Scenario: College Database**

Entities: Student, Course, Professor, Department

Relationships:

- Student **enrolls in** Course (M:N)
- Professor **teaches** Course (1:N)
- Professor **belongs to** Department (N:1)
- Student **belongs to** Department (N:1)




## 4.8 Extended ER Features

**Generalization**

- Bottom-up approach
- Multiple lower-level entities are combined into a higher-level entity
- Example: Car, Truck, Bike → generalized into Vehicle

**Specialization**

- Top-down approach
- A higher-level entity is divided into lower-level sub-entities based on characteristics
- Example: Employee → specialized into Manager, Engineer, Clerk

**Aggregation**

- A relationship treated as a higher-level entity
- Used when a relationship needs to participate in another relationship
- Example: Employee **works on** Project is an aggregation, and this aggregation is **managed by** a Manager

# PART 5 — RELATIONAL MODEL

## 5.1 What is the Relational Model?

The relational model organizes data into tables (called relations). Each table has rows (called tuples) and columns (called attributes). It was proposed by Edgar F. Codd at IBM in 1970.

## 5.1 What is the Relational Model?

The relational model organizes data into tables (called relations). Each table has rows (called tuples) and columns (called attributes). It was proposed by Edgar F. Codd at IBM in 1970.

## 5.2 Terminology

| Relational Model Term | Common Term |
|---|---|
| Relation | Table |
| Tuple | Row / Record |
| Attribute | Column / Field |
| Domain | Set of allowed values for an attribute |
| Degree | Number of attributes (columns) in a table |
| Cardinality | Number of tuples (rows) in a table |

**Example Table: Student**

| student_id | name | age | city |
|---|---|---|---|
| 101 | Rahul | 21 | Delhi |
| 102 | Priya | 22 | Mumbai |
| 103 | Arjun | 20 | Pune |

- **Degree** = 4 (four columns)
- **Cardinality** = 3 (three rows)
- **Domain of age** = positive integers (typically 1-100)

## 5.3 Properties of a Relation

- Each relation has a unique name
- Each attribute has a distinct name within the relation
- Values in a column must be from the same domain
- Each tuple is unique (no two rows are identical)
- The order of tuples does not matter
- The order of attributes does not matter
- Each cell contains exactly one atomic (indivisible) value — this is the First Normal Form requirement

# PART 6 — KEYS IN DBMS

Keys are the most important concept for both database design and interviews.

## 6.1 Super Key

A super key is any attribute or set of attributes that can uniquely identify a tuple in a relation.

**Example: Employee table**

(employee_id) -- uniquely identifies
(employee_id, name) -- still unique (superset)
(employee_id, name, email) -- still unique (superset)
(email) -- uniquely identifies (if all emails are unique)
All of the above are super keys. A super key may have redundant attributes.

## 6.2 Candidate Key

A candidate key is a **minimal super key** — a super key from which no attribute can be removed while still maintaining uniqueness.

**Example:**

(employee_id) -- Candidate Key (minimal, unique)
(email) -- Candidate Key (if emails are unique)
(employee_id, name) -- NOT a candidate key (employee_id alone is sufficient)
A table can have multiple candidate keys.

## 6.3 Primary Key

A primary key is the candidate key chosen by the database designer to uniquely identify tuples in a relation.

**Rules:**

- Must be unique — no two rows can have the same primary key value
- Must not be NULL — primary key values are always required
- Should be stable — ideally, primary key values never change

**Example:**

- Must be unique — no two rows can have the same primary key value
- Must not be NULL — primary key values are always required
- Should be stable — ideally, primary key values never change

**Example:**

```
CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

Here student_id is chosen as the primary key.

## 6.4 Alternate Key

A candidate key that was not chosen as the primary key is called an alternate key.
**Example:** If student_id is the primary key, then email (which is also unique) is an alternate key.

## 6.5 Foreign Key

A foreign key is an attribute (or set of attributes) in one table that references the primary key of another table. It establishes a relationship between two tables.

**Rules:**

- The value in the foreign key column must either match a primary key value in the referenced table, OR be NULL
- This is called **Referential Integrity**

**Example:**

```
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100)
);
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Here, dept_id in the Employee table is a foreign key referencing Department.

## 6.6 Composite Key

A composite key is a primary key that consists of two or more attributes. Used when no single attribute can uniquely identify a tuple.

**Example: Enrollment table**

```
CREATE TABLE Enrollment (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id)
);
```

Here, neither student_id alone nor course_id alone is unique. But together they are unique.

## 6.7 Surrogate Key

A surrogate key is an artificially generated key — typically an auto-incremented integer — with no business meaning. It is used purely to uniquely identify rows.

**Example:**

```
CREATE TABLE Order (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE
);
```

order_id is a surrogate key. It has no meaning in the real world — it exists only to uniquely identify each order.

# PART 7 — SQL (STRUCTURED QUERY LANGUAGE)

## 7.1 What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases. It was developed by IBM in the 1970s and is standardized by ISO/ANSI.
SQL is declarative — you tell the database *what* you want, not *how* to get it.

## 7.2 Categories of SQL Commands

SQL Commands
├── DDL (Data Definition Language) -- Structure of the database

## 7.2 Categories of SQL Commands

SQL Commands
├── DDL (Data Definition Language) -- Structure of the database
├── DML (Data Manipulation Language) -- Data inside the tables
├── DCL (Data Control Language) -- Access and permissions
├── TCL (Transaction Control Language)-- Transaction management
└── DQL (Data Query Language) -- Querying data (SELECT)

## 7.3 DDL — Data Definition Language

DDL commands define, modify, or remove the structure of database objects.

**CREATE**

```
-- Create a database
CREATE DATABASE college_db;
-- Use the database
USE college_db;
-- Create a table
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 18),
    dept_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

**ALTER**

```
-- Add a new column
ALTER TABLE Student ADD phone VARCHAR(15);
-- Modify a column's data type
ALTER TABLE Student MODIFY age SMALLINT;
-- Rename a column
ALTER TABLE Student RENAME COLUMN phone TO contact_number;
-- Drop a column
ALTER TABLE Student DROP COLUMN contact_number;
```

**DROP**

```
-- Drop a table (removes structure AND all data permanently)
DROP TABLE Student;
-- Drop a database
DROP DATABASE college_db;
```

**TRUNCATE**

-- Removes all rows from a table but keeps the structure
TRUNCATE TABLE Student;

**Difference: DROP vs TRUNCATE vs DELETE**

| Feature | DROP | TRUNCATE | DELETE |
|---|---|---|---|
| Removes structure | Yes | No | No |
| Removes data | Yes | Yes | Yes (specific rows) |
| Can use WHERE | No | No | Yes |
| Can be rolled back | No | No (generally) | Yes |
| Speed | Fast | Fast | Slow (row-by-row) |
| Resets AUTO_INCREMENT | N/A | Yes | No |

## 7.4 DML — Data Manipulation Language

DML commands manipulate the actual data stored in tables.

```
INSERT
-- Insert a single row
INSERT INTO Student (student_id, name, email, age, dept_id)
VALUES (101, 'Rahul Sharma', 'rahul@example.com', 21, 10);
-- Insert multiple rows at once
INSERT INTO Student (student_id, name, email, age, dept_id)
VALUES
    (102, 'Priya Patel', 'priya@example.com', 22, 10),
    (103, 'Arjun Singh', 'arjun@example.com', 20, 20),
    (104, 'Sneha Gupta', 'sneha@example.com', 23, 30);
UPDATE
-- Update a specific row
UPDATE Student
SET email = 'rahul.sharma@example.com'
WHERE student_id = 101;
-- Update multiple columns
UPDATE Student
SET age = 22, dept_id = 20
WHERE student_id = 101;
-- Update all rows (dangerous — no WHERE clause)
UPDATE Student SET age = age + 1;

DELETE
-- Delete a specific row
DELETE FROM Student WHERE student_id = 104;
-- Delete rows meeting a condition
DELETE FROM Student WHERE age < 18;
-- Delete all rows (keeps table structure)
DELETE FROM Student;
```

## 7.5 DQL — SELECT Queries

SELECT is the most-used command in SQL.

```
DELETE FROM Student;
```

## 7.5 DQL — SELECT Queries

SELECT is the most-used command in SQL.

```
Basic SELECT
-- Select all columns
SELECT * FROM Student;
-- Select specific columns
SELECT name, email FROM Student;
-- Add an alias
SELECT name AS student_name, age AS student_age FROM Student;
WHERE Clause
-- Single condition
SELECT * FROM Student WHERE age > 20;
-- Multiple conditions with AND
SELECT * FROM Student WHERE age > 20 AND dept_id = 10;
-- Multiple conditions with OR
SELECT * FROM Student WHERE dept_id = 10 OR dept_id = 20;
-- NOT
SELECT * FROM Student WHERE NOT dept_id = 30;
-- BETWEEN
SELECT * FROM Student WHERE age BETWEEN 18 AND 22;
-- IN
SELECT * FROM Student WHERE dept_id IN (10, 20, 30);
-- LIKE (pattern matching)
SELECT * FROM Student WHERE name LIKE 'R%';     -- starts with R
SELECT * FROM Student WHERE name LIKE '%a';     -- ends with a
SELECT * FROM Student WHERE name LIKE '%rah%';  -- contains rah
SELECT * FROM Student WHERE name LIKE '_r%';    -- second character is r
-- IS NULL
SELECT * FROM Student WHERE email IS NULL;
SELECT * FROM Student WHERE email IS NOT NULL;
ORDER BY
-- Sort ascending (default)
SELECT * FROM Student ORDER BY age;
-- Sort descending
SELECT * FROM Student ORDER BY age DESC;
-- Sort by multiple columns
SELECT * FROM Student ORDER BY dept_id ASC, age DESC;
LIMIT and OFFSET
-- Get only the first 5 records
SELECT * FROM Student LIMIT 5;
-- Pagination: skip 10 records, get the next 5
SELECT * FROM Student LIMIT 5 OFFSET 10;
```

## 7.6 Aggregate Functions

Aggregate functions perform calculations on a set of values and return a single value.

```
-- COUNT: number of rows
SELECT COUNT(*) FROM Student;
SELECT COUNT(email) FROM Student; -- counts only non-NULL emails
-- SUM: total of a numeric column
SELECT SUM(age) FROM Student;
-- AVG: average value
SELECT AVG(age) FROM Student;
-- MAX and MIN
SELECT MAX(age) FROM Student;
SELECT MIN(age) FROM Student;
```

## 7.7 GROUP BY and HAVING

**GROUP BY** groups rows that have the same value in specified columns and allows aggregate functions to be applied to each group.

```
-- Count students in each department
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id;
-- Average age per department
SELECT dept_id, AVG(age) AS avg_age
FROM Student
GROUP BY dept_id;
HAVING filters groups after GROUP BY (like WHERE but for groups).
-- Only show departments with more than 5 students
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

**Important Distinction:**

| Clause | When it filters | Works on |
|---|---|---|
| WHERE | Before grouping | Individual rows |
| HAVING | After grouping | Groups (aggregated results) |

```
-- WHERE filters rows first, then GROUP BY groups them, then HAVING filters groups
SELECT dept_id, COUNT(*) AS student_count
FROM Student
WHERE age >= 18
GROUP BY dept_id
HAVING COUNT(*) > 3;
```

## 7.8 SQL Execution Order

This is extremely important to understand why certain queries fail.

FROM

## 7.8 SQL Execution Order

This is extremely important to understand why certain queries fail.

FROM
  |
JOIN
  |
WHERE
  |
GROUP BY
  |
HAVING
  |
SELECT
  |
DISTINCT
  |
ORDER BY
  |
LIMIT / OFFSET

This means you cannot use a SELECT alias in a WHERE clause, because WHERE is evaluated before SELECT.

## 7.9 JOINS

Joins combine rows from two or more tables based on a related column.

**Setup for examples:**

```
CREATE TABLE Department (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(100)
);
INSERT INTO Department VALUES (10, 'Computer Science'), (20, 'Electronics'), (30, 'Mechanical');
CREATE TABLE Employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(100),
   dept_id INT
);
INSERT INTO Employee VALUES
   (1, 'Rahul', 10),
   (2, 'Priya', 20),
   (3, 'Arjun', 10),
   (4, 'Sneha', 99);  -- dept_id 99 does NOT exist in Department
```

**INNER JOIN**

Returns only rows that have matching values in both tables.

```
SELECT e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
```

**Result:**

```
emp_name | dept_name
---------|-----------------
Rahul    | Computer Science
Priya    | Electronics
Arjun    | Computer Science
```

Note: Sneha is excluded because dept_id 99 has no match in Department.

**LEFT JOIN (LEFT OUTER JOIN)**

Returns all rows from the left table. If no match in the right table, fills with NULL.

```
SELECT e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

**Result:**

```
emp_name | dept_name
---------|-----------------
Rahul    | Computer Science
Priya    | Electronics
Arjun    | Computer Science
Sneha    | NULL
```

Sneha is included, but her dept_name is NULL.

**RIGHT JOIN (RIGHT OUTER JOIN)**

Returns all rows from the right table. If no match in the left table, fills with NULL.

```
SELECT e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

**Result:**

```
emp_name | dept_name
---------|-----------------
Rahul    | Computer Science
Priya    | Electronics
Arjun    | Computer Science
NULL     | Mechanical
```

Mechanical department has no employees, so emp_name is NULL.

**FULL OUTER JOIN**

Returns all rows from both tables. If no match on either side, fills with NULL. (MySQL does not

NULL | Mechanical
Mechanical department has no employees, so emp_name is NULL.

**FULL OUTER JOIN**
Returns all rows from both tables. If no match on either side, fills with NULL. (MySQL does not support FULL OUTER JOIN directly — use UNION of LEFT and RIGHT JOIN.)

```
SELECT e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

**CROSS JOIN**
Returns the Cartesian product — every row from the first table paired with every row from the second table.

```
SELECT e.emp_name, d.dept_name
FROM Employee e
CROSS JOIN Department d;
```

If Employee has 4 rows and Department has 3 rows, the result has 12 rows. Use carefully.

**SELF JOIN**
A table joined with itself. Useful for hierarchical data.

```
-- Find each employee and their manager
-- Assume Employee table has a manager_id column referencing emp_id
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Employee e
JOIN Employee m ON e.manager_id = m.emp_id;
```

## 7.10 Subqueries

A subquery is a query nested inside another query.

```
-- Find students older than the average age
SELECT name, age
FROM Student
WHERE age > (SELECT AVG(age) FROM Student);
-- Find employees in departments located in Delhi
SELECT emp_name
FROM Employee
WHERE dept_id IN (
   SELECT dept_id FROM Department WHERE location = 'Delhi'
);
-- Correlated subquery: executed once for each row of the outer query
SELECT emp_name, salary
FROM Employee e
WHERE salary > (
   SELECT AVG(salary)
   FROM Employee
   WHERE dept_id = e.dept_id
);
```

## 7.11 Views

A view is a virtual table created from a SELECT query. It does not store data physically — it is a saved query.

```
-- Create a view
CREATE VIEW CS_Students AS
SELECT student_id, name, email
FROM Student
WHERE dept_id = 10;
-- Use the view like a table
SELECT * FROM CS_Students;
-- Update the view definition
CREATE OR REPLACE VIEW CS_Students AS
SELECT student_id, name, email, age
FROM Student
WHERE dept_id = 10;
-- Drop the view
DROP VIEW CS_Students;
```

**Why use views?**

- Security: Expose only specific columns to certain users
- Simplicity: Hide complex joins behind a simple view name
- Consistency: Multiple applications query the same view rather than duplicating complex SQL

## 7.12 Stored Procedures

A stored procedure is a precompiled set of SQL statements stored in the database that can be executed by name.

```
-- Create a stored procedure
DELIMITER $$
CREATE PROCEDURE GetStudentsByDept(IN dept INT)
BEGIN
   SELECT student_id, name, age
   FROM Student
   WHERE dept_id = dept;
```

```
CREATE PROCEDURE GetStudentsByDept(IN dept INT)
BEGIN
  SELECT student_id, name, age
  FROM Student
  WHERE dept_id = dept;
END $$
DELIMITER ;
-- Call the stored procedure
CALL GetStudentsByDept(10);
```

**Advantages:**

- Reduced network traffic (one call instead of multiple queries)
- Code reuse
- Security (execute permission without table access)
- Precompiled so faster than ad-hoc queries

## 7.13 Triggers

A trigger is a special stored procedure that automatically executes when a specific event (INSERT, UPDATE, DELETE) occurs on a table.

```
-- Log every student deletion to an audit table
CREATE TABLE Student_Audit (
  audit_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT,
  deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
DELIMITER $$
CREATE TRIGGER before_student_delete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
  INSERT INTO Student_Audit (student_id)
  VALUES (OLD.student_id);
END $$
DELIMITER ;
```

**Trigger events:** BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, AFTER DELETE

## 7.14 DCL — Data Control Language

DCL commands manage user permissions and access.

```
-- Grant SELECT and INSERT permission on Student table to user 'john'
GRANT SELECT, INSERT ON college_db.Student TO 'john'@'localhost';
-- Grant all privileges
GRANT ALL PRIVILEGES ON college_db.* TO 'admin'@'localhost';
-- Remove permissions
REVOKE INSERT ON college_db.Student FROM 'john'@'localhost';
```

## 7.15 TCL — Transaction Control Language

TCL commands manage transactions. (Covered in detail in the Transactions section below.)

```
START TRANSACTION;
UPDATE Account SET balance = balance - 5000 WHERE account_id = 'A';
UPDATE Account SET balance = balance + 5000 WHERE account_id = 'B';
COMMIT;   -- Save both changes permanently
-- OR in case of error:
ROLLBACK; -- Undo all changes since START TRANSACTION
```

# PART 8 — CONSTRAINTS

Constraints are rules applied to columns or tables to enforce data integrity.

| Constraint | Purpose | Example |
|---|---|---|
| PRIMARY KEY | Uniquely identifies each row; NOT NULL + UNIQUE | student_id INT PRIMARY KEY |
| FOREIGN KEY | Enforces referential integrity between tables | FOREIGN KEY (dept_id) REFERENCES Department(dept_id) |
| UNIQUE | Ensures all values in a column are different | email VARCHAR(100) UNIQUE |
| NOT NULL | Ensures a column cannot have a NULL value | name VARCHAR(100) NOT NULL |
| CHECK | Ensures values meet a specific condition | age INT CHECK (age >= 18) |
| DEFAULT | Sets a default value if none is provided | status VARCHAR(20) DEFAULT 'active' |

```
CREATE TABLE Product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) CHECK (price > 0),
  category VARCHAR(50) DEFAULT 'General',
  sku VARCHAR(50) UNIQUE,
  supplier_id INT,
  FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
```

);

# PART 9 — NORMALIZATION

## 9.1 What is Normalization?

Normalization is the process of organizing a relational database to reduce **data redundancy** (storing the same data in multiple places) and **data anomalies** (insert, update, and delete anomalies).
It was introduced by Edgar F. Codd alongside the relational model.

## 9.2 Why Normalization?

**Anomalies in an unnormalized table:**
Consider this unnormalized Order table:

| order_id | customer_name | customer_phone | product_name | product_price | quantity |
|---|---|---|---|---|---|
| 1 | Rahul | 9876543210 | Laptop | 60000 | 1 |
| 2 | Rahul | 9876543210 | Mouse | 500 | 2 |
| 3 | Priya | 8765432109 | Keyboard | 1000 | 1 |

**Insert Anomaly:** Cannot add a new product to the catalog without creating a fake order.
**Update Anomaly:** If Rahul's phone number changes, you must update it in every row where Rahul appears. If you miss one row, the data is inconsistent.
**Delete Anomaly:** If you delete order 3 (Priya's order), you lose all information about the Keyboard product.

## 9.3 First Normal Form (1NF)

**Rule:** Every column must contain atomic (indivisible) values. No repeating groups or arrays.
**Violation:**

| student_id | name | courses |
|---|---|---|
| 101 | Rahul | DBMS, OS, CN |

The courses column is not atomic.
**After 1NF:**

| student_id | name | course |
|---|---|---|
| 101 | Rahul | DBMS |
| 101 | Rahul | OS |
| 101 | Rahul | CN |

Now each cell has exactly one value.

## 9.4 Second Normal Form (2NF)

**Prerequisite:** Must already be in 1NF.
**Rule:** Every non-key attribute must be **fully functionally dependent** on the entire primary key. No partial dependency (where a non-key attribute depends on only part of a composite primary key).
**This only applies when the primary key is composite.**
**Violation:**
Table: Enrollment(student_id, course_id, student_name, course_name, grade) Primary Key: (student_id, course_id)

- student_name depends only on student_id — partial dependency
- course_name depends only on course_id — partial dependency
- grade depends on both student_id AND course_id — full dependency (OK)

**After 2NF — Split into 3 tables:**

```
Student(student_id, student_name)        -- student_name depends on student_id
Course(course_id, course_name)           -- course_name depends on course_id
Enrollment(student_id, course_id, grade)  -- grade depends on both
```

## 9.5 Third Normal Form (3NF)

**Prerequisite:** Must already be in 2NF.
**Rule:** There must be no **transitive dependency** — a non-key attribute should not depend on another non-key attribute.
**Violation:**
Table: Employee(emp_id, emp_name, dept_id, dept_name) Primary Key: emp_id

- dept_id depends on emp_id (OK)
- dept_name depends on dept_id (not on emp_id) — this is a transitive dependency

**After 3NF:**

```
Employee(emp_id, emp_name, dept_id)
Department(dept_id, dept_name)
```

## 9.6 Boyce-Codd Normal Form (BCNF)

**After 3NF:**
Employee(emp_id, emp_name, dept_id)
Department(dept_id, dept_name)

## 9.6 Boyce-Codd Normal Form (BCNF)

**Prerequisite:** Must already be in 3NF.
**Rule:** For every non-trivial functional dependency X → Y, X must be a super key.
BCNF is a stricter version of 3NF. Most tables that are in 3NF are also in BCNF, but there are edge cases involving overlapping candidate keys.
**Violation Example:**
CourseInstructor(student_id, course, instructor)

- Each student takes a course taught by only one instructor
- Each instructor teaches only one course
- A course can have multiple instructors

Functional dependencies:

- (student_id, course) → instructor
- instructor → course (an instructor teaches only one course)

The dependency instructor → course violates BCNF because instructor is not a super key.
**After BCNF:**
Instructor(instructor, course)
StudentInstructor(student_id, instructor)

## 9.7 Fourth Normal Form (4NF)

**Rule:** No multi-valued dependencies. A table should not have two or more independent multi-valued facts about an entity.
**Violation:**
Employee(emp_id, skill, hobby) — where skills and hobbies are independent of each other.
**After 4NF:**
EmployeeSkill(emp_id, skill)
EmployeeHobby(emp_id, hobby)

## 9.8 Functional Dependency

A functional dependency X → Y means that for any two tuples with the same value of X, they must also have the same value of Y. X functionally determines Y.
**Example:** In a Student table, student_id → name because knowing the student_id tells you exactly which student's name it is.
**Types:**

- **Trivial FD:** Y is a subset of X. Example: {student_id, name} → student_id
- **Non-trivial FD:** Y is not a subset of X. Example: student_id → name
- **Partial Dependency:** A non-key attribute depends on part of a composite key
- **Transitive Dependency:** A → B and B → C, therefore A → C transitively

## 9.9 Normalization Summary

| Normal Form | Eliminates |
|---|---|
| 1NF | Multi-valued and composite attributes (ensures atomicity) |
| 2NF | Partial dependencies |
| 3NF | Transitive dependencies |
| BCNF | Anomalies where non-superkeys determine attributes |
| 4NF | Multi-valued dependencies |

# PART 10 — TRANSACTIONS AND ACID PROPERTIES

## 10.1 What is a Transaction?

A transaction is a logical unit of work that consists of one or more database operations that must all succeed or all fail together.
**Classic Example: Bank Transfer**
Transfer Rs. 5000 from Account A to Account B:
Step 1: Read balance of A
Step 2: balance_A = balance_A - 5000
Step 3: Write balance of A
Step 4: Read balance of B
Step 5: balance_B = balance_B + 5000
Step 6: Write balance of B
If the system crashes after Step 3 but before Step 6, Rs. 5000 is deducted from A but never credited to B. Money disappears. This is exactly what transactions prevent.

Step 6: Write balance of B
If the system crashes after Step 3 but before Step 6, Rs. 5000 is deducted from A but never credited to B. Money disappears. This is exactly what transactions prevent.

## 10.2 ACID Properties

ACID is an acronym that defines the four properties that every database transaction must guarantee.

**A — Atomicity**
A transaction is treated as a single indivisible unit. Either ALL operations in the transaction succeed and are committed, or ALL operations fail and are rolled back. There is no partial completion.
**Managed by:** Transaction Manager / Recovery Manager
**Example:**

```
START TRANSACTION;
UPDATE Account SET balance = balance - 5000 WHERE account_id = 'A';
UPDATE Account SET balance = balance + 5000 WHERE account_id = 'B';
COMMIT;
```

If the second UPDATE fails, the ROLLBACK automatically undoes the first UPDATE.

**C — Consistency**
A transaction brings the database from one valid state to another valid state. All data integrity rules, constraints, and cascades are maintained before and after the transaction.
**Example:** If a rule says "account balance cannot go below zero," then a transaction attempting to withdraw more than the available balance will be rejected. The database remains consistent.

**I — Isolation**
Concurrent transactions execute as if they are the only transaction running. The intermediate state of a transaction is not visible to other transactions.
**Isolation levels** (from weakest to strongest):

1. **Read Uncommitted** — A transaction can read data that another transaction has modified but not yet committed (dirty reads possible)
2. **Read Committed** — A transaction can only read data that has been committed (prevents dirty reads)
3. **Repeatable Read** — Once a transaction reads data, that data cannot be changed by another transaction until this transaction completes (prevents non-repeatable reads)
4. **Serializable** — Strictest level. Transactions are completely isolated from each other, executed one after another

**D — Durability**
Once a transaction is committed, the changes are permanent and survive system failures (power loss, crashes). This is achieved through write-ahead logging (WAL).
**How it works:** Before data is written to disk, the operation is first written to a transaction log. If the system crashes, the log is used to replay or undo operations during recovery.

## 10.3 Transaction States




## 10.4 Concurrency Problems

When multiple transactions run simultaneously without proper isolation, these problems occur:
**Dirty Read** Transaction T2 reads data that T1 has modified but not yet committed. If T1 rolls back, T2 has read data that never officially existed.

```
T1: UPDATE salary = 80000 WHERE emp_id = 1;
T2: SELECT salary WHERE emp_id = 1;  -- reads 80000 (uncommitted)
T1: ROLLBACK;  -- salary goes back to original
```

T2 has now used a phantom value
**Non-Repeatable Read** T1 reads a row. T2 updates that row and commits. T1 reads the same row again and gets a different value.
**Phantom Read** T1 queries rows matching a condition. T2 inserts a new row that matches the condition and commits. T1 runs the same query again and gets an extra row that was not there before.
**Lost Update** T1 and T2 both read a value, both update it, and T1's update is overwritten by T2.

# PART 11 — CONCURRENCY CONTROL

## 11.1 What is Concurrency Control?

Concurrency control ensures that simultaneous execution of transactions in a multi-user environment does not cause data inconsistency. The goal is to allow maximum concurrency while maintaining the correctness of data.

## 11.2 Lock-Based Protocols

**Shared Lock (S-Lock / Read Lock)**

- Allows a transaction to READ a data item
- Multiple transactions can hold a shared lock on the same item simultaneously
- A transaction holding a shared lock cannot write until the lock is upgraded

**Exclusive Lock (X-Lock / Write Lock)**

- Allows a transaction to both READ and WRITE a data item
- Only one transaction can hold an exclusive lock on an item at a time
- No other transaction can read or write while the exclusive lock is held

| | **Shared Lock Requested** | **Exclusive Lock Requested** |
|---|---|---|
| Shared Lock Held | Compatible | Not Compatible |
| Exclusive Lock Held | Not Compatible | Not Compatible |

## 11.3 Two-Phase Locking (2PL)

Two-Phase Locking is a concurrency control protocol that guarantees serializability.
**Phase 1: Growing Phase** A transaction acquires all the locks it needs. It can acquire new locks but cannot release any lock.
**Phase 2: Shrinking Phase** A transaction releases its locks. It cannot acquire any new locks.
The point where a transaction acquires its last lock is called the **lock point**.
**Problem with 2PL:** It can cause deadlocks.

## 11.4 Deadlock

A deadlock occurs when two or more transactions are waiting for each other to release locks, creating a circular dependency where none can proceed.
**Example:**
T1 holds lock on A, waiting for B
T2 holds lock on B, waiting for A
--> Neither can proceed. Deadlock.
**Deadlock Detection:** Build a wait-for graph. If a cycle exists, a deadlock exists. Resolve by aborting one transaction (the victim).
**Deadlock Prevention:** Assign priorities to transactions. Use timestamp-based schemes:

- **Wait-Die:** Older transaction waits; younger transaction dies (rolls back)
- **Wound-Wait:** Older transaction wounds (preempts) younger; younger waits

## 11.5 Timestamp-Based Concurrency Control

Each transaction is assigned a unique timestamp when it starts. Transactions are ordered by their timestamps.
**Rules:**

- If T1 starts before T2, timestamp(T1) < timestamp(T2)
- If T tries to read an item that has been written by a newer transaction, T is rolled back
- If T tries to write an item that has been read or written by a newer transaction, T is rolled back

# PART 12 — INDEXING

## 12.1 What is an Index?

An index is a data structure that improves the speed of data retrieval operations on a table at the cost of additional storage space and slower write operations.
Think of it like the index at the back of a textbook — instead of reading every page to find a topic,

## 12.1 What is an Index?

An index is a data structure that improves the speed of data retrieval operations on a table at the cost of additional storage space and slower write operations.
Think of it like the index at the back of a textbook — instead of reading every page to find a topic, you look up the topic in the index and jump directly to the page.

## 12.2 How Indexes Work

Without an index, a query does a **full table scan** — reading every row.
With an index on a column, the database uses the index data structure (typically a B-tree) to find the exact location of matching rows in O(log n) time.

## 12.3 Types of Indexes

**Primary Index**

- Built automatically on the primary key
- Data is physically sorted by the primary key
- Also called a clustered index

**Secondary Index (Non-Clustered Index)**

- Built on non-primary key columns
- Points to the physical location of the data
- A table can have multiple secondary indexes

**Clustered Index**

- Determines the physical order of data in the table
- Only one clustered index per table (the data can only be sorted one way)
- In most databases, the primary key creates the clustered index

**Non-Clustered Index**

- A separate structure from the table data
- Contains the indexed column values and pointers to the actual data rows
- Multiple non-clustered indexes can exist on a table

**Composite Index**

- An index on multiple columns
- Useful for queries that filter on multiple columns

```
-- Create an index on the name column of Student
CREATE INDEX idx_student_name ON Student(name);
-- Create a composite index on dept_id and age
CREATE INDEX idx_dept_age ON Student(dept_id, age);
-- Drop an index
DROP INDEX idx_student_name ON Student;
```

## 12.4 Dense vs Sparse Index

**Dense Index**

- An index entry for every record in the data file
- Faster search, but more storage
- Required for non-ordered (non-primary) attributes

**Sparse Index**

- An index entry only for some records (e.g., one per data block)
- Less storage, but the search requires scanning within the block
- Only possible when data is physically sorted by the indexed attribute

## 12.5 B-Tree Index

Most relational databases use B-trees (Balanced Trees) as the underlying data structure for indexes.
**Properties:**

- Self-balancing tree
- All leaf nodes are at the same level
- Supports range queries efficiently (BETWEEN, <, >)
- O(log n) for search, insert, and delete

## 12.6 Hash Index

Uses a hash function to map key values to positions in a hash table.
**Properties:**

- O(1) lookup for exact match queries
- Does NOT support range queries
- Example use: Exact lookups on primary keys

## 12.7 When to Use and Not Use Indexes

**Use indexes when:**

- Column is frequently used in WHERE, JOIN ON, ORDER BY, or GROUP BY
- Column has high cardinality (many distinct values)
- Table is large and reads are frequent

**Avoid indexes when:**

- Table is small (full scan is faster than index lookup overhead)
- Column has very low cardinality (e.g., a gender column with values M/F — index barely helps)
- Table has frequent INSERT, UPDATE, DELETE (index maintenance slows writes)

# PART 13 — FILE ORGANIZATION IN DATABASES

## 13.1 Heap File Organization

- Records are stored in no particular order
- New records are inserted at the end
- Retrieval requires a full scan
- Best for: Small tables or bulk loading

## 13.2 Sequential File Organization

- Records are stored sorted by a key field
- Efficient for range queries
- Insertion is expensive (maintaining order)
- Best for: Reporting and sequential processing

## 13.3 Hashing File Organization

- Records are placed using a hash function on a key field
- O(1) retrieval for exact key matches
- Poor for range queries
- Best for: Exact lookups

## 13.4 Clustered File Organization

- Related records from multiple tables stored together on the same disk block
- Reduces I/O for JOIN operations
- Used when two tables are frequently joined

# PART 14 — RELATIONAL ALGEBRA

Relational Algebra is a procedural query language that operates on relations (tables) and returns a relation as a result. It forms the theoretical foundation of SQL.

## 14.1 Basic Operations

**Selection (sigma — σ)** Filters rows based on a condition. Equivalent to SQL's WHERE clause.
σ(age > 20)(Student)
-- Returns all rows from Student where age > 20
**Projection (pi — π)** Selects specific columns. Equivalent to SELECT clause.
π(name, email)(Student)
-- Returns only name and email columns from Student
**Union (U)** Combines rows from two relations. Both must have the same attributes. Removes duplicates.
Students_Delhi ∪ Students_Mumbai
**Intersection (∩)** Returns rows present in both relations.
**Difference (–)** Returns rows in the first relation but not in the second.
**Cartesian Product (×)** Returns all possible combinations of rows from two relations. Equivalent to CROSS JOIN.
**Rename (ρ)** Renames a relation or its attributes.

## 14.2 Derived Operations

**Join (⋈)** Natural Join returns rows with matching values on common attributes (inner join behavior).
**Division (÷)** Used for "for all" queries. Example: Find students who have enrolled in ALL courses.

# PART 15 — NoSQL DATABASES

# PART 15 — NoSQL DATABASES

## 15.1 What is NoSQL?

NoSQL (Not Only SQL) databases are non-relational databases designed for large-scale, distributed data storage where flexibility, horizontal scalability, and high performance matter more than strict schema enforcement.

## 15.2 Why NoSQL?

Relational databases have limitations:

- Scaling vertically (adding more RAM/CPU to one machine) is expensive and has a ceiling
- Rigid schemas make it hard to handle unstructured or rapidly changing data
- Some data is naturally non-tabular (documents, graphs, time series)

## 15.3 Types of NoSQL Databases

**Document Store**

- Data stored as JSON/BSON documents
- Schema-less — each document can have different fields
- Best for: Content management, user profiles, catalogs
- Example: MongoDB

```
{
  "_id": "user_101",
  "name": "Rahul Sharma",
  "age": 25,
  "address": {
    "city": "Delhi",
    "pin": "110001"
  },
  "skills": ["Python", "SQL", "MongoDB"]
}
```

**Key-Value Store**

- Data stored as key-value pairs
- Extremely fast for read/write
- Best for: Session data, caching, shopping carts
- Examples: Redis, DynamoDB

```
SET user:101:name "Rahul Sharma"
GET user:101:name
```

**Column-Family Store (Wide-Column)**

- Data stored in columns grouped into column families
- Optimized for reading/writing large amounts of data across many rows
- Best for: Time-series data, analytics, IoT data
- Examples: Apache Cassandra, HBase

**Graph Database**

- Data stored as nodes (entities) and edges (relationships)
- Optimized for traversing complex relationships
- Best for: Social networks, recommendation engines, fraud detection
- Examples: Neo4j, Amazon Neptune

## 15.4 CAP Theorem

The CAP Theorem states that in a distributed database system, it is impossible to simultaneously guarantee all three of the following:

**C — Consistency** Every read receives the most recent write or an error.

**A — Availability** Every request receives a response (not necessarily the most recent data).

**P — Partition Tolerance** The system continues operating even when network partitions (communication failures) occur between nodes.

In a real distributed system, partition tolerance is not optional — network failures happen. So the practical choice is between:

- **CP (Consistency + Partition Tolerance):** MongoDB, HBase — prefer to return an error over stale data
- **AP (Availability + Partition Tolerance):** Cassandra, CouchDB — prefer to return possibly stale data over no response

## 15.5 RDBMS vs NoSQL — Summary

| Feature | RDBMS | NoSQL |
|---|---|---|
| Schema | Fixed, predefined | Flexible, dynamic |
| Query language | SQL | Varies by database |
| Scalability | Vertical (scale up) | Horizontal (scale out) |
| ACID compliance | Full ACID | Eventual consistency (usually) |
| Best for | Structured data, complex queries | Unstructured data, high volume, speed |
| Examples | MySQL, PostgreSQL, Oracle | MongoDB, Redis, Cassandra |

| | | |
|---|---|---|
| ACID compliance | Full ACID | Eventual consistency (usually) |
| Best for | Structured data, complex queries | Unstructured data, high volume, speed |
| Examples | MySQL, PostgreSQL, Oracle | MongoDB, Redis, Cassandra |

# PART 16 — DATABASE RECOVERY

## 16.1 What is Recovery?

Recovery is the process of restoring the database to a consistent state after a failure.

**Types of Failures:**

- Transaction failure — logical error, deadlock
- System failure — power loss, OS crash (data in volatile memory is lost; disk data survives)
- Media failure — disk crash (permanent data loss)

## 16.2 Write-Ahead Logging (WAL)

Before any change is written to the database, it is first written to a log file. This ensures that even if the system crashes mid-transaction, the log can be replayed or undone.

**Log record format:**

[Transaction ID, Operation, Object, Old Value, New Value]

## 16.3 Checkpointing

A checkpoint is a point in the log where the DBMS guarantees that all committed transactions have been written to disk. During recovery, the system only needs to process logs after the last checkpoint, reducing recovery time.

## 16.4 Recovery Techniques

**Undo Logging** If a transaction fails, all changes made by that transaction are undone (rolled back to the old values).

**Redo Logging** If a committed transaction's changes are not yet on disk (still in buffer), they are redone (written to disk from the log).

**Undo/Redo Logging** Combines both — uncommitted changes are undone, and committed changes that are not on disk are redone.

# PART 17 — HASHING IN DATABASES

## 17.1 Static Hashing

A fixed number of buckets. A hash function maps a key to a bucket number.

bucket = hash(key) % number_of_buckets

**Problem:** If data grows, many keys will hash to the same bucket (overflow), causing performance degradation.

## 17.2 Dynamic Hashing (Extendible Hashing)

The hash table grows dynamically as data grows. Buckets split when they overflow, and the directory doubles in size. This avoids the overflow problem of static hashing.

# PART 18 — STORED PROCEDURES vs FUNCTIONS vs TRIGGERS

## PART 18 — STORED PROCEDURES vs FUNCTIONS vs TRIGGERS

| Feature | Stored Procedure | Function | Trigger |
|---|---|---|---|
| Returns value | Optional (OUT params) | Must return a value | Does not return a value |
| Called how | Explicitly (CALL) | Used in SQL expressions | Automatically on event |
| Can modify data | Yes | Generally no (read-only) | Yes |
| Transaction control | Yes (COMMIT, ROLLBACK) | No | Limited |
| Use case | Business logic, workflows | Computations, formatting | Auditing, auto-updates |

# PART 19 — ADVANCED SQL CONCEPTS

## 19.1 Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing them into a single group like GROUP BY does.

```
-- Rank employees by salary within each department
SELECT emp_name, dept_id, salary,
   RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS salary_rank
FROM Employee;
-- Running total of sales
SELECT order_date, amount,
   SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM Sales;
-- Row number for pagination
SELECT emp_name, salary,
   ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM Employee;
```

**Common Window Functions:**

- ROW_NUMBER() — unique sequential number for each row
- RANK() — rank with gaps (1, 1, 3 if two rows tie)
- DENSE_RANK() — rank without gaps (1, 1, 2 if two rows tie)
- NTILE(n) — divides rows into n equal buckets
- LAG(col, offset) — access value from a previous row
- LEAD(col, offset) — access value from a following row

## 19.2 Common Table Expressions (CTEs)

A CTE is a temporary, named result set defined within a query. It makes complex queries readable.

```
WITH DeptStats AS (
   SELECT dept_id, AVG(salary) AS avg_salary
   FROM Employee
   GROUP BY dept_id
)
SELECT e.emp_name, e.salary, d.avg_salary
FROM Employee e
JOIN DeptStats d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_salary;
```

**Recursive CTE** — used for hierarchical data:

```
WITH RECURSIVE OrgChart AS (
   -- Base case: CEO (no manager)
   SELECT emp_id, emp_name, manager_id, 0 AS level
   FROM Employee
   WHERE manager_id IS NULL

   UNION ALL

   -- Recursive case: all employees with a manager in the result set
   SELECT e.emp_id, e.emp_name, e.manager_id, o.level + 1
   FROM Employee e
   JOIN OrgChart o ON e.manager_id = o.emp_id
)
SELECT * FROM OrgChart;
```

## 19.3 CASE Expression

```
SELECT emp_name, salary,
   CASE
      WHEN salary > 100000 THEN 'Senior'
```

```
    WHEN salary > 50000 THEN 'Mid-Level'
    ELSE 'Junior'
  END AS grade
FROM Employee;
```

## 19.4 UNION vs UNION ALL

```
-- UNION removes duplicates
SELECT name FROM TableA
UNION
SELECT name FROM TableB;
-- UNION ALL keeps all rows including duplicates (faster)
SELECT name FROM TableA
UNION ALL
SELECT name FROM TableB;
```

## 19.5 EXISTS vs IN

```
-- EXISTS: more efficient when subquery returns many rows
SELECT emp_name FROM Employee e
WHERE EXISTS (
  SELECT 1 FROM Orders o WHERE o.emp_id = e.emp_id
);
-- IN: simpler syntax, works well for small subquery results
SELECT emp_name FROM Employee
WHERE emp_id IN (SELECT emp_id FROM Orders);
```

# PART 20 — DATABASE DESIGN

## Steps in Designing a Database

**Step 1: Requirements Analysis** Understand what data needs to be stored and the operations that need to be performed.
**Step 2: Conceptual Design** Create an ER diagram identifying entities, attributes, and relationships.
**Step 3: Logical Design** Convert the ER diagram into a relational schema (set of tables). Apply normalization to remove redundancy.
**Step 4: Physical Design** Decide on file organization, indexes, partitioning, and storage details.
**Step 5: Implementation** Write the DDL statements to create the database and tables.

## Complete Real-World Example: E-Commerce Database

```
-- Customers
CREATE TABLE Customer (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Product Categories
CREATE TABLE Category (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL
);
-- Products
CREATE TABLE Product (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(200) NOT NULL,
  description TEXT,
  price DECIMAL(10, 2) NOT NULL CHECK (price > 0),
  stock_quantity INT DEFAULT 0 CHECK (stock_quantity >= 0),
  category_id INT,
  FOREIGN KEY (category_id) REFERENCES Category(category_id)
);
-- Orders
CREATE TABLE Orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
-- Orders
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) DEFAULT 'Pending',
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
-- Order Items (junction table for Orders and Products)
CREATE TABLE OrderItem (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
-- Addresses
CREATE TABLE Address (
    address_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    street VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(100),
    pin_code VARCHAR(10),
    is_default BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
```

# PART 21 — 20 MOST ASKED DBMS INTERVIEW QUESTIONS

**Q1. What is the difference between a database and a DBMS?**
A database is an organized collection of data stored electronically. A DBMS is the software system used to create, manage, and interact with that database. The database is the data itself; the DBMS is the tool that manages it. Example: MySQL is a DBMS; the set of tables and data it manages is the database.

**Q2. What are the ACID properties? Explain each with an example.**
ACID stands for Atomicity, Consistency, Isolation, and Durability.

- **Atomicity**: A bank transfer either fully completes (debit + credit) or is fully rolled back. No partial updates.
- **Consistency**: Before and after the transfer, the total money in the system remains the same. All rules and constraints are satisfied.
- **Isolation**: If two people are transferring money simultaneously, each transaction behaves as if it is the only one running.
- **Durability**: Once the bank confirms the transfer, the record survives even a system crash.

**Q3. What is the difference between DELETE, TRUNCATE, and DROP?**

- DELETE removes specific rows based on a WHERE condition. It is a DML command, logs every row deletion, and can be rolled back. It does not reset AUTO_INCREMENT.
- TRUNCATE removes all rows from a table without logging individual rows. It is faster than DELETE, cannot be rolled back in most databases, and resets AUTO_INCREMENT. It keeps the table structure.
- DROP removes the entire table — both structure and data — permanently. It cannot be rolled back.

**Q4. What are the different types of joins? Explain with examples.**

- **INNER JOIN**: Returns rows that have matching values in both tables.
- **LEFT JOIN**: Returns all rows from the left table; unmatched rows from the right table appear as NULL.
- **RIGHT JOIN**: Returns all rows from the right table; unmatched rows from the left table appear as NULL.
- **FULL OUTER JOIN**: Returns all rows from both tables; unmatched rows on either side appear as NULL.
- **CROSS JOIN**: Returns the Cartesian product — every row from the left table paired with every row from the right table.
- **SELF JOIN**: Joins a table with itself — used for hierarchical data like employee-manager

as NULL.
- **CROSS JOIN**: Returns the Cartesian product — every row from the left table paired with every row from the right table.
- **SELF JOIN**: Joins a table with itself — used for hierarchical data like employee-manager relationships.

**Q5. What is normalization? What are 1NF, 2NF, and 3NF?**
Normalization is the process of structuring a database to reduce data redundancy and prevent anomalies.
- **1NF**: Every column must contain atomic (indivisible) values. No repeating groups.
- **2NF**: Must be in 1NF. Every non-key attribute must be fully functionally dependent on the entire primary key (no partial dependencies — applies to composite keys).
- **3NF**: Must be in 2NF. No transitive dependencies — a non-key attribute should not depend on another non-key attribute.

**Q6. What is the difference between a primary key and a unique key?**

| | **Primary Key** | **Unique Key** |
|---|---|---|
| Uniqueness | Must be unique | Must be unique |
| NULL values | Not allowed | Allowed (one NULL in most databases) |
| Count per table | Only one | Multiple allowed |
| Creates index | Clustered index | Non-clustered index |

**Q7. What is a foreign key and what is referential integrity?**
A foreign key is an attribute in one table that references the primary key of another table. Referential integrity is the constraint that ensures every foreign key value either matches a primary key value in the referenced table or is NULL. It prevents orphan records — records in the child table that have no corresponding record in the parent table.

**Q8. What is the difference between WHERE and HAVING?**
Both filter rows, but:
- **WHERE** filters individual rows before any grouping occurs. It cannot use aggregate functions.
- **HAVING** filters groups after GROUP BY is applied. It can use aggregate functions.

```
-- WHERE: filter rows before grouping
SELECT dept_id, COUNT(*) FROM Employee
WHERE age > 25
GROUP BY dept_id
HAVING COUNT(*) > 3;
-- HAVING: filter groups after grouping
```

**Q9. What is a deadlock? How is it detected and resolved?**
A deadlock occurs when two or more transactions are each waiting for a lock held by the other, creating a circular wait where none can proceed.
**Detection**: Build a wait-for graph. If a cycle is detected in the graph, a deadlock exists.
**Resolution**: The DBMS selects a victim transaction (usually the one that has done the least work or holds the fewest locks) and aborts it. The other transaction then proceeds.
**Prevention**: Use timestamp-based protocols (Wait-Die or Wound-Wait) to impose an ordering on transactions and prevent circular waits.

**Q10. What is an index? What are its advantages and disadvantages?**
An index is a data structure (usually a B-tree) that provides fast data retrieval by creating a quick lookup path to rows in a table.
**Advantages**: Dramatically speeds up SELECT queries, especially on large tables. Improves JOIN, ORDER BY, and GROUP BY performance.
**Disadvantages**: Consumes additional storage. Slows down INSERT, UPDATE, and DELETE operations because the index must also be updated. Too many indexes on a table degrades write performance.

**Q11. What is the difference between a clustered and a non-clustered index?**
A **clustered index** determines the physical order of data storage in the table. There can only be one clustered index per table because the data can only be physically sorted one way. In most RDBMS, the primary key creates a clustered index automatically.
A **non-clustered index** is a separate structure from the table data. It contains the indexed column values and pointers to the actual data rows. A table can have multiple non-clustered indexes.

**Q12. What is a view? What are its advantages?**
A view is a virtual table created by a stored SELECT query. It does not store data — it runs the underlying query each time the view is accessed.
**Advantages**: Security (expose only specific columns), simplicity (hide complex joins), consistency (multiple apps use the same view), and logical independence.

**Q13. What are the different isolation levels in SQL?**
From weakest to strongest:
1. **Read Uncommitted**: Can read uncommitted data (dirty reads possible)
2. **Read Committed**: Can only read committed data (prevents dirty reads)
3. **Repeatable Read**: Data read during a transaction cannot be changed by others until the transaction ends (prevents non-repeatable reads)
4. **Serializable**: Full isolation — transactions execute as if they are the only transaction (prevents phantom reads)

Higher isolation = fewer concurrency problems but lower throughput.

**Q14. What is the difference between OLTP and OLAP?**

| **OLTP** | **OLAP** |
|---|---|

**Q14. What is the difference between OLTP and OLAP?**

| | OLTP | OLAP |
|---|---|---|
| Full form | Online Transaction Processing | Online Analytical Processing |
| Purpose | Day-to-day transactions | Data analysis and reporting |
| Query type | Short, simple INSERT/UPDATE/DELETE | Complex, long-running SELECT with aggregations |
| Data volume | Current, operational | Historical, large volumes |
| Examples | Banking app, e-commerce checkout | Business intelligence, data warehouse |
| Database | Normalized (3NF) | Denormalized (star/snowflake schema) |

**Q15. What is the difference between SQL and NoSQL?**
SQL databases are relational, use structured schemas, support ACID transactions, and scale vertically. They are best for structured data with complex relationships (banking, ERP, CRM).
NoSQL databases are non-relational, schema-flexible, typically favor availability and partition tolerance (CAP theorem), and scale horizontally. They are best for unstructured, high-volume, rapidly changing data (social feeds, product catalogs, real-time analytics).

**Q16. What is a stored procedure and how is it different from a function?**
A stored procedure is a precompiled set of SQL statements stored in the database and called explicitly. It can perform INSERT, UPDATE, DELETE, and return multiple result sets. It may or may not return a value.
A function must return a single value. It is used within SQL expressions. In most databases, functions cannot perform DML operations (insert/update/delete). Procedures can manage transactions; functions generally cannot.

**Q17. Explain the concept of transactions and the states of a transaction.**
A transaction is a logical unit of work consisting of one or more operations that must succeed or fail together.
States: Active (executing), Partially Committed (all operations done, awaiting commit), Committed (permanently saved), Failed (error occurred), Aborted (rolled back to previous state).

**Q18. What is BCNF and when does 3NF differ from BCNF?**
BCNF (Boyce-Codd Normal Form) states that for every non-trivial functional dependency X → Y, X must be a super key.
3NF allows non-super key attributes on the left side of a functional dependency if the right side is part of some candidate key. BCNF is stricter.
3NF and BCNF differ when there are overlapping candidate keys. A relation can be in 3NF but not BCNF if a non-super key attribute determines part of a candidate key.

**Q19. What is the CAP theorem and how does it apply to database selection?**
The CAP Theorem states that a distributed system can guarantee at most two of the three: Consistency, Availability, and Partition Tolerance. Since network partitions are unavoidable in distributed systems, the real choice is between Consistency (CP systems like MongoDB) and Availability (AP systems like Cassandra).
When selecting a database: for a banking application where consistency is critical, choose a CP system. For a social media feed where a few seconds of stale data is acceptable, choose an AP system for high availability.

**Q20. Write a query to find the second highest salary from an Employee table.**
**Method 1: Using LIMIT and OFFSET**

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

**Method 2: Using a subquery**

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

**Method 3: Using DENSE_RANK (most robust — handles ties correctly)**

```
WITH RankedSalaries AS (
  SELECT salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
)
SELECT salary
FROM RankedSalaries
WHERE rnk = 2;
```

**Method 4: For the Nth highest salary (generalized)**

```
WITH RankedSalaries AS (
  SELECT salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
)
SELECT salary
FROM RankedSalaries
WHERE rnk = N;  -- Replace N with the desired rank
```